Boolean Algebra over Trapdoor Sets: A Practical Framework for Privacy-Preserving Set Operations with Probabilistic Guarantees

Alexander Towell
Department of Computer Science
Southern Illinois University Edwardsville
Email: atowell@siue.edu

***Abstract*—We study the *trapdoor set*: a set whose elements are opaque, deterministic one-way trapdoors[1], on which an untrusted party computes union, intersection, and difference directly over the opaque encodings. Membership is answered by equality of encodings, returning a plaintext boolean. The construction never recovers plaintext; what it exposes is the *equality predicate* among encodings—the leakage profile of deterministic and searchable encryption. Its only source of error is a hash collision: every operation has no false negatives and a false-positive rate bounded by the collision probability $|S|\,2^{-n}$, with no other approximation.**

Two parameters fix the construction's position on its trade-offs. A *representation* choice trades space for accuracy: a literal set of element trapdoors keeps every operation collision-bounded in $O(k)$ space, whereas a compact fixed-width encoding (the image of a single-hash Bloom filter) saves space but makes intersection *structurally* approximate—wrong even absent collisions—and cannot preserve complement. Indeed, under either representation the finite trapdoor sets form a *generalized* Boolean algebra (a Boolean ring without unit) rather than a Boolean algebra: the complement of a finite set is not finitely representable. A *homophony* parameter $K(x)$, the number of encodings per value, fixes confidentiality: $K = 1$ (deterministic) opens the equality channel and admits the untrusted-side algebra studied here, while $K > 1$ closes that channel and yields the opaque, trusted-decode regime of the cipher-map framework that this construction instantiates as its online case.

We give systematic error-propagation rules for composed operations, the corresponding size-dependent false-positive analysis, and a reference implementation backing the claims. Because operations reduce to hashing and set operations rather than homomorphic evaluation or multi-round interaction, the trapdoor set avoids the overhead of fully homomorphic encryption and secure multi-party computation; we position it against those alternatives and, most closely, against searchable symmetric encryption.

***Index Terms*—cryptographic hash functions, trapdoor data structures, privacy-preserving computation, approximate algorithms, probabilistic data structures**

[1]We use "trapdoor" to mean a one-way encoding keyed by a secret, not a trapdoor permutation in the standard cryptographic sense. The key k enables creation of encodings $T_k(v) = H(k\|v)$ but does not permit recovery of inputs—there is no efficient inversion.

I. Introduction

The proliferation of cloud computing and data analytics has created an urgent need for privacy-preserving computational frameworks that enable operations on sensitive data without exposing the underlying values. Traditional approaches to this problem fall into two categories: cryptographic techniques such as fully homomorphic encryption (FHE) [1] and secure multi-party computation (MPC) [2], and statistical techniques such as differential privacy [3]. While powerful, these approaches often suffer from computational overhead that limits their practical deployment.

We study the *trapdoor set*: a set whose elements are deterministic one-way encodings $T_k(v) = H(k\|v)$. Because equal values produce equal encodings, an untrusted party can compute union, intersection, difference, and membership directly over the opaque elements without ever recovering a plaintext value. The cost of this efficiency is precise and bounded: the construction exposes the *equality predicate* among encodings—the leakage profile of deterministic and searchable symmetric encryption—and its only computational error is a hash collision.

The core insight is that many applications can tolerate controlled error rates in exchange for efficiency and privacy. Cryptographic hash functions serve as one-way transformations: sensitive data is mapped into a hash domain where equality testing is preserved probabilistically while original values remain computationally hidden. All subsequent operations work exclusively on hash values, never requiring access to plaintext.

Two parameters place the construction on its trade-offs. The *representation* parameter trades space for accuracy: a literal set of element trapdoors makes every operation collision-bounded in $O(k)$ space, while a compact fixed-width encoding (the image of a single-hash Bloom filter) saves space but renders intersection structurally approximate and complement non-preservable. The *homophony* parameter $K(x)$—the number of encodings per value—fixes confidentiality: $K = 1$ (deterministic) opens the equality channel that makes the untrusted-

side algebra possible, while $K > 1$ closes it, recovering the opaque, trusted-decode regime of the cipher-map framework [4] that this construction instantiates as its online case. We are careful about the algebraic structure this yields: with union, intersection, difference, and symmetric difference but no representable complement, the finite trapdoor sets form a *generalized* Boolean algebra (a Boolean ring without unit), not a Boolean algebra. Error-propagation rules for composed operations follow Bernoulli type theory [5].

A. Motivating Example

Consider a healthcare consortium where multiple hospitals need to identify patients enrolled in overlapping clinical trials without revealing their complete patient lists. Traditional approaches would require either: (1) a trusted third party to perform the intersection, violating privacy requirements, (2) homomorphic encryption with prohibitive computational costs, or (3) complex MPC protocols requiring multiple rounds of communication.

Using the trapdoor set, each hospital can:

1) Transform patient identifiers using a shared cryptographic hash function
2) Perform set intersection directly on hash values
3) Obtain results with explicit error bounds (e.g., hash collision rate $< 2^{-256}$)
4) Complete the operation non-interactively, at the cost of hashing and set operations, avoiding the homomorphic-evaluation or multi-round overhead of FHE or MPC

B. Contributions

This paper makes the following contributions:

- **The trapdoor set as a value type**: we define the trapdoor set and characterize its algebra precisely—union, intersection, difference, and symmetric difference that are collision-bounded (no false negatives; false-positive rate $|S|\,2^{-n}$), forming a generalized Boolean algebra (a Boolean ring without unit) rather than a Boolean algebra, since complement is not finitely representable.
- **A representation trade-off**: we contrast the literal set (exact up to collision, $O(k)$ space) with the compact single-hash bit-vector (space-saving, but with a structurally approximate intersection and a complement we prove cannot be preserved).
- **A confidentiality dial**: we show the homophony parameter $K(x)$ interpolates between the deterministic, equality-leaking regime studied here ($K = 1$) and the opaque, trusted-decode regime of the cipher-map framework ($K > 1$), situating the trapdoor set as the online instantiation of that framework.
- **Error propagation and false-positive analysis**: we give closed-form error-propagation rules for composed operations and a size-dependent false-positive analysis for both representations.
- **Reference implementation**: we provide the `TrapdoorSet` type backing these claims with unit, property, and integration tests.

II. Background and Threat Model

A. Cryptographic Hash Functions

A cryptographic hash function $H : \{0,1\}^* \to \{0,1\}^n$ maps arbitrary-length inputs to fixed-size outputs satisfying the standard properties: preimage resistance ($O(2^n)$ to invert), second preimage resistance ($O(2^n)$ to find a collision for a given input), and collision resistance ($O(2^{n/2})$ to find any collision). The trapdoor set exploits these properties to create one-way transformations that preserve equality testing probabilistically while preventing recovery of original values.

B. Approximate Data Structures

Approximate data structures trade perfect accuracy for improved space or time complexity. The canonical example is the Bloom filter [6], which supports membership queries with false positives but no false negatives. The trapdoor set builds upon these well-established techniques, adding cryptographic privacy through hash transformations while maintaining explicit error bounds.

Definition 1 (Bernoulli Boolean)**.** *A* Bernoulli Boolean $\mathcal{B}\langle\mathbb{B}\rangle$ *is an observed boolean* $\tilde{b}$ *with error rates* (α, β) *where:*

- $\alpha = \mathbb{P}[\tilde{b} = \mathsf{true} \mid b = \mathsf{false}]$ *(false positive rate)*
- $\beta = \mathbb{P}[\tilde{b} = \mathsf{false} \mid b = \mathsf{true}]$ *(false negative rate)*

We distinguish second-order *Bernoulli Booleans where* $\alpha \neq \beta$ *(asymmetric errors, as in Bloom filters and hash-based membership) from* first-order *where* $\alpha = \beta$ *(symmetric errors). The trapdoor set's membership test is a second-order Bernoulli Boolean with single-comparison* $\alpha = 2^{-n}$ *(hash collision) and* $\beta = 0$*; against a set of size* $|S|$ *the membership false-positive rate is* $|S|\,2^{-n}$ *by a union bound.*

Definition 2 (Confusion Matrix)**.** *The error behavior of a Bernoulli Boolean is fully characterized by its* confusion matrix*:*

$$Q = \begin{pmatrix} 1-\alpha & \alpha \\ \beta & 1-\beta \end{pmatrix}$$

where $Q_{ij} = \mathbb{P}[\tilde{b} = j \mid b = i]$ *gives the probability of observing* j *given latent value* i*. Key properties:*

- ***Identity matrix*** *(*$\alpha = \beta = 0$*): perfect observation, no privacy*
- ***Rank-1 matrix*** *(*$\alpha + \beta = 1$*): complete information loss, optimal privacy*
- ***Rank-2 matrix*** *(*$\alpha + \beta < 1$*): information preserved, privacy/utility trade-off*

For a single comparison with $\alpha = 2^{-n}$ *and* $\beta = 0$*, the confusion matrix is nearly identity—high utility, but query results are observable through the equality channel (Section III-C).*

Throughout, we distinguish *latent* values (b, S, f) from *observed* approximations $(\tilde{b}, \tilde{S}, \tilde{f})$ carrying explicit error rates.

C. Threat Model

We consider an honest-but-curious adversary model where:

- Participants follow the protocol correctly but attempt to learn additional information

- The adversary has access to hash values but cannot invert the hash function
- The adversary may have auxiliary information about the data distribution
- The cryptographic hash function is modeled as a random oracle

We explicitly exclude:

- Malicious adversaries who deviate from the protocol
- Side-channel attacks on the implementation
- Quantum adversaries (though post-quantum hash functions could be used)

III. System Design

A. Core Abstractions

The trapdoor set is built around three core abstractions that compose to enable complex privacy-preserving operations:

1) Hash-Trapdoor Values: A hash-trapdoor value is a one-way transformation of sensitive data: equality testing is preserved probabilistically (FPR equals hash collision probability, e.g., 2^{-256} for SHA-256) while original values cannot be recovered. Note that this bounds *collision errors*, not privacy—privacy depends on input entropy and resistance to dictionary attacks.

2) Approximate Values: All operations in the trapdoor set return approximate values carrying both the computed result and its false positive and false negative rates (α, β), with confidence $1 - \alpha - \beta$ when $\alpha + \beta \leq 1$.

3) Set Operations: The trapdoor set supports the operations of a generalized Boolean algebra over its opaque elements:

- Union $A \cup B$, intersection $A \cap B$, difference $A \setminus B$, and symmetric difference $A \triangle B$, computed as set operations on the element encodings;
- Membership $x \in A$ by encoding equality, with false-positive rate bounded by the hash-collision probability and no false negatives.

There is no complement operation: the complement of a finite set is not finitely representable (Proposition 10). Symmetric difference makes the finite trapdoor sets a Boolean ring without unit—$\triangle$ is addition and $\cap$ is multiplication—which is convenient for aggregation. The compact bit-vector representation (Section IV-B) offers a syntactic complement via bitwise $\neg$, but it does not preserve set complement.

B. Error Propagation

A key aspect of the trapdoor set is systematic error propagation through operations. For Boolean operations, we derive tight bounds:

Theorem 3 (Union Error Bound)**.** *For sets A and B with false positive rates α_A, α_B and false negative rates β_A, β_B:*

$$\alpha_{A \cup B} \leq \alpha_A + \alpha_B - \alpha_A \cdot \alpha_B \tag{1}$$

$$\beta_{A \cup B} = \beta_A \cdot \beta_B \tag{2}$$

Proof. Dual to Theorem 4 by De Morgan's laws: $A \cup B = \overline{\bar{A} \cap \bar{B}}$. Complement swaps error rates ($\alpha \leftrightarrow \beta$), and intersection multiplies false positive rates while accumulating false negative rates via inclusion-exclusion. Equivalently: a false positive in the union occurs if *either* set reports a false positive (independent events, yielding the inclusion-exclusion bound $\alpha_A + \alpha_B - \alpha_A \alpha_B$), while a false negative requires *both* sets to miss the element (yielding the product $\beta_A \cdot \beta_B$). □

Theorem 4 (Intersection Error Bound)**.** *For sets A and B with false positive rates α_A, α_B and false negative rates β_A, β_B:*

$$\alpha_{A \cap B} = \alpha_A \cdot \alpha_B \tag{3}$$

$$\beta_{A \cap B} \leq \beta_A + \beta_B - \beta_A \cdot \beta_B \tag{4}$$

Proof. For intersection, a false positive requires *both* sets to report false positives (independent events). A false negative occurs if *either* set reports a false negative for an element truly in the intersection. □

Note that intersection *reduces* false positive rates (multiplicative) while union *increases* them (sub-additive).

Theorem 5 (Boolean Error Composition)**.** *For independent approximate Booleans $\tilde{b}_1, \tilde{b}_2$ with error rates (α_1, β_1) and (α_2, β_2):*

$$\neg \tilde{b}: \quad (\beta, \alpha) \text{ (rates swap)} \tag{5}$$

$$\tilde{b}_1 \wedge \tilde{b}_2: \quad (\alpha_1 \alpha_2,\ \beta_1 + \beta_2 - \beta_1 \beta_2) \tag{6}$$

$$\tilde{b}_1 \vee \tilde{b}_2: \quad (\alpha_1 + \alpha_2 - \alpha_1 \alpha_2,\ \beta_1 \beta_2) \tag{7}$$

Proof. Negation swaps the meaning of false positive and false negative. For conjunction, a false positive requires both operands to err (independent), while a false negative occurs if either errs. Disjunction is dual by De Morgan's laws. □

Remark 6 (Bernoulli Model Foundation)**.** *The closed-form error rates for AND, OR, NOT, and their compositions above are consequences of the Bernoulli model's element-wise independence axiom: each element's membership error is an independent channel. Formally, the joint confusion matrix for a t-element operation factors as a Kronecker (tensor) product of per-element confusion matrices $Q^{\otimes t}$, so that composed error rates reduce to scalar arithmetic on (α, β) pairs. This factorization is what makes the error propagation rules tractable—without independence, composed error bounds would require the full joint distribution over all elements.*

Corollary 7 (Composition Accumulation)**.** *For t-fold composition of independent Bernoulli Booleans each with FPR α:*

- ***t-fold OR (union)**: $\alpha_{total} = 1 - (1 - \alpha)^t \approx t\alpha$ for small α.*
- ***t-fold AND (intersection)**: $\alpha_{total} = \alpha^t$ (FPR decreases exponentially).*

Dually, for FNR β: t-fold AND accumulates as $\beta_{total} = 1 - (1 - \beta)^t$, and t-fold OR reduces as $\beta_{total} = \beta^t$.

C. Privacy: Non-Recovery and the Equality Channel

The construction's privacy has two distinct mechanisms: *trapdoor representation* through one-way encodings, by which nothing is recovered, and an *equality channel* that deterministic encoding opens, by which structural relations among elements become observable.

1) Trapdoor Representation Layer: The hash transformation $T_k(v) = H(k\|v)$ creates a *trapdoor representation*: inspecting a hash value reveals nothing about the underlying data beyond what can be learned through defined operations. This provides:

- **Preimage resistance**: Cannot recover v from $T_k(v)$ without exhaustive search
- **Key-dependent hiding**: Without knowledge of k, an adversary cannot verify membership guesses
- **Representation uniformity**: Hash values appear uniformly random, hiding structural properties of inputs

2) The Equality Channel: Because encoding is deterministic, encodings can be compared for equality, and this is what the untrusted side exploits:

- **Membership**: $x \in S$ is decided by encoding equality, returning a plaintext boolean with false-positive rate at most $|S|\,2^{-n}$ and no false negatives;
- **Set operations**: union, intersection, difference, and symmetric difference are computed directly on encodings, exact up to the same collision bound;
- **What an observer sees**: the equality relations among encodings—which coincide, which queries hit—but no plaintext value.

Remark 8 (What leaks: the equality channel)**.** *The trapdoor never reveals a plaintext value, but a party holding the encoded set learns the* equality pattern *among encodings: which elements coincide, and whether a queried encoding is present. This is the leakage profile of deterministic and searchable symmetric encryption [7], and it is a different axis from decoding—equality leaks without any preimage being broken. It is the price of letting the untrusted side run the algebra itself, and it is exactly what the homophony parameter K controls: $K = 1$ opens this channel, while $K > 1$ closes it at the cost of moving evaluation behind a trusted decoder.*

3) Hash Values as Bernoulli Set Carriers: The `HashValue` type with bitwise operations ($\wedge$, $\vee$, $\oplus$, $\neg$) forms the carrier for Bernoulli set operations. Each hash equality test is a second-order Bernoulli Boolean:

$$\alpha = 2^{-n} \quad \text{(hash collision probability for } n\text{-bit hash)} \tag{8}$$

$$\beta = 0 \quad \text{(no false negatives for hash equality)} \tag{9}$$

This asymmetry ($\alpha > 0$, $\beta = 0$) propagates through composed operations via the error composition rules above.

D. Architecture

The construction follows a layered architecture as shown in Figure 1:

Applications (PSI, Analytics, Aggregation)
Operations Layer (Similarity, Cardinality Estimation)
Set Layer (Boolean Algebra, Symmetric Difference)
Core Primitives (Hash-Trapdoor Values, Approximate Types)
Cryptographic Layer (SHA-256, BLAKE2b, etc.)

Fig. 1: Layered architecture

IV. Mathematical Foundations

A. Trapdoor Sets and Their Algebra

Fix a value universe X and a deterministic keyed one-way encoding $T_k : X \to \{0,1\}^n$, $T_k(v) = H(k\|v)$. The *trapdoor set* of a finite $A \subseteq X$ is the literal set of its element encodings,

$$\tau(A) := \{\, T_k(a) : a \in A \,\} \subseteq \{0,1\}^n.$$

An untrusted party holding $\tau(A)$ and $\tau(B)$ computes union, intersection, difference, and symmetric difference as ordinary set operations on the byte strings, and answers membership by the equality test $T_k(x) \in \tau(A)$, returning a plaintext boolean. None of these operations uses the secret key, and none recovers a plaintext value.

Proposition 9 (Collision-bounded set algebra)**.** *Condition on T_k being injective on the values in question; the complementary event is a hash collision, of probability at most $\binom{|A|+|B|}{2}2^{-n}$. Under injectivity, τ commutes with all four operations exactly,*

$$\tau(A) \odot \tau(B) = \tau(A \odot B), \qquad \odot \in \{\cup, \cap, \setminus, \triangle\},$$

and $T_k(x) \in \tau(A) \iff x \in A$. Consequently membership has no false negatives and a false-positive rate at most $|A|\,2^{-n}$, and this collision bound is the only *error in the set operations: there is no structural approximation.*

Proof. T_k is a function, so $a \in A \Rightarrow T_k(a) \in \tau(A)$; there are no false negatives. When T_k is injective on $A \cup B$, the map $a \mapsto T_k(a)$ is a bijection onto $\tau(A \cup B)$ that carries each Boolean operation on subsets of $A \cup B$ to the corresponding operation on images, giving exact commutation. A membership false positive requires $T_k(x) = T_k(a)$ for some $a \in A$ with $x \notin A$; a union bound over $a \in A$ gives $|A|\,2^{-n}$. The injectivity failure probability is the birthday bound over the $\leq |A| + |B|$ values involved. □

Proposition 10 (Algebraic structure)**.** *Over an unbounded universe X (e.g. $X = \{0,1\}^*$), the finite trapdoor sets under $(\cup, \cap, \setminus, \triangle, \emptyset)$ form a* generalized *Boolean algebra: a relatively complemented distributive lattice with least element $\emptyset$ but no greatest element—equivalently a Boolean ring without unit, with $\triangle$ as addition and $\cap$ as multiplication. They are* not *a Boolean algebra, because complement is absent: $X \setminus A$ is infinite and has no finite trapdoor-set representation. Over a finite universe X the greatest element $\tau(X)$ exists and complement becomes $\tau(X) \setminus \tau(A)$, recovering a bounded Boolean algebra—at the cost of materializing $\tau(X)$, the encoding of the entire universe, which is practical only for small X.*

The literal representation stores $|A|$ encodings and keeps every operation collision-bounded. When space is the binding

constraint, a trapdoor set can be compressed to a fixed-width bit vector, at the cost of a *structural* approximation the literal form avoids. We develop that compact representation next; it is the object of the homomorphism and the complement-non-preservation result that follow.

B. A Compact Representation: Bit-Vector Encoding

The compact representation maps a trapdoor set to a single m-bit vector by OR-ing its element hashes. It is a homomorphism between two Boolean algebras: the *set algebra* $A := (\mathcal{P}(\{0,1\}^*), \cup, \cap, \neg, \emptyset, \{0,1\}^*)$ over arbitrary bit strings, and the *bit vector algebra* $B := (\{0,1\}^m, \vee, \wedge, \neg, 0^m, 1^m)$ of m-bit vectors with bitwise operations. Unlike the literal set, this map has a syntactic top 1^m and a syntactic complement (bitwise $\neg$); we will see that the latter does not correspond to set complement (Theorem 22), so the compact form only appears to be a Boolean algebra.

Definition 11 (Trapdoor Homomorphism)**.** *The homomorphism $F : A \to B$ is defined as:*

$$F(\beta) := \begin{cases} H(\beta) & \beta \in \{0,1\}^* \\ \vee & \beta = \cup \\ \wedge & \beta = \cap \\ \neg & \beta = \neg_A \\ 0^m & \beta = \emptyset \\ 1^m & \beta = \{0,1\}^* \end{cases}$$

where $H : \{0,1\}^* \to \{0,1\}^m$ is a cryptographic hash function. For a set $S = \{x_1, \ldots, x_k\}$, this extends to $F(S) = H(x_1) \vee H(x_2) \vee \cdots \vee H(x_k)$.

***Caveat on intersection**: The mapping $\cap \mapsto \wedge$ satisfies $F(A) \wedge F(B) \supseteq F(A \cap B)$, not equality. Extra 1-bits arise when an element of $A \setminus B$ and an element of $B \setminus A$ both set the same bit position, producing false positives in the intersection encoding. The union mapping $\cup \mapsto \vee$ is exact.*

Remark 12 (Bloom Filter Connection)**.** *The construction presented here is mathematically equivalent to a Bloom filter with a single hash function ($k = 1$). The contribution of this paper is not the construction itself, which is well known, but the algebraic analysis: the characterization as a Boolean algebra homomorphism, the precise error propagation formulas for composed operations, and the proof that complement is not preserved.*

Theorem 13 (One-Wayness of Trapdoor Homomorphism)**.** *The homomorphism F is one-way for two independent reasons:*

1) ***Non-injectivity**: F is not injective. Since $\{0,1\}^*$ is countably infinite and $\{0,1\}^m$ has only 2^m elements, by the pigeonhole principle, countably infinitely many inputs map to each output.*
2) ***Preimage resistance**: The hash function H is a cryptographic hash that resists preimage attacks. Given $y \in \{0,1\}^m$, finding any x such that $H(x) = y$ requires $O(2^m)$ operations, while computing $H(x)$ from x is efficient.*

Remark 14 (Bit-Rate Asymptotics)**.** *The bit-rate (bits per element) required to achieve a target false positive rate ε for a set of s elements in an m-bit vector can be derived as follows. From the FPR formula $\varepsilon = \alpha(s)^m$ where $\alpha(s) = 1 - 2^{-(s+1)}$ is the per-bit occupancy probability, solving for m gives:*

$$m = \frac{\log_2 \varepsilon}{\log_2 \alpha(s)}$$

For large s, $\alpha(s) \to 1$ and writing $\delta = 2^{-(s+1)}$, we have:

$$\log_2(1 - \delta) \approx \frac{-\delta}{\ln 2}$$

so the required bit-vector length is:

$$m \approx \frac{-\log_2(\varepsilon) \cdot \ln 2}{\delta} = -\log_2(\varepsilon) \cdot \ln 2 \cdot 2^{s+1}$$

and the bit-rate m/s grows exponentially with s.

Remark 15 (Absolute Efficiency)**.** *The* absolute efficiency *of the bit-vector representation is:*

$$\mathsf{Eff}(s) = -s \log_2 \alpha(s)$$

which tends to 0 exponentially as $s \to \infty$ (since $\log_2 \alpha(s) \to 0$ while s grows linearly), making this construction practical only for small sets (typically $s \leq 20$). For larger sets, the two-level hashing scheme of Section IV-E is required.

C. Security Analysis

We separate two guarantees: *non-recovery* of plaintext values, and *confidentiality* against an adversary who observes the encodings and the equality relations among them. Preimage resistance provides the first; it does not provide the second.

Theorem 16 (Non-Recovery)**.** *Let $H : \{0,1\}^* \to \{0,1\}^n$ be modeled as a random oracle. Under the random oracle model, recovering x from $H(k\|x)$ requires expected $O(2^n)$ hash evaluations when k is secret and uniformly random. No efficient algorithm can determine the input x from the hash output with non-negligible probability.*

Proof. In the random oracle model, $H(k\|x)$ is uniformly distributed over $\{0,1\}^n$ and independent of x from the adversary's perspective (since k is unknown). Each hash evaluation tests one candidate input, and the probability of matching the target output is 2^{-n} per evaluation. After q queries, the success probability is at most $q \cdot 2^{-n}$, which is negligible for q polynomial in n. □

Remark 17 (Confidentiality is governed by the equality channel, not preimage hardness)**.** *Theorem 16 prevents recovering x from $T_k(x)$, but says nothing about what the encodings leak* collectively. *Under deterministic encoding ($K = 1$), an adversary observing the encoded set and a stream of queries learns the partition of values by equality and their frequencies—the leakage profile of deterministic and searchable symmetric encryption [7]. The appropriate confidentiality measure is*

therefore quantitative information flow: the entropy of the observed encoding distribution relative to the prior, degraded precisely by the equality channel and improved by homophony ($K > 1$). Preimage hardness bounds non-recovery; it is not a confidentiality bound.

D. Size-Dependent False Positives in the Compact Representation

For the literal trapdoor set, membership false positives are governed solely by the collision bound of Proposition 9, independent of layout. The compact bit-vector representation is different: because elements share a fixed-width vector, its false-positive rate depends on how densely the set fills it, which we now derive.

Theorem 18 (Membership False Positive Rate)**.** *For a set S of size k represented as an m-bit vector, the false positive rate for membership queries is:*

$$\varepsilon_{\in}(k, m) = \left(1 - 2^{-(k+1)}\right)^m$$

For large k, this approximates to $\varepsilon_{\in} \approx e^{-m \cdot 2^{-(k+1)}}$.

Proof. After k insertions, each bit position is set to 1 with probability $1 - 2^{-k}$. For a random element $x \notin S$ to test positive, every bit position where $h(x)$ is 1 must also be 1 in $F(S)$. Since $h(x)$ has each bit set with probability $\frac{1}{2}$, the probability that bit j does not refute membership is:

$$\Pr[\neg(h(x)_j = 1 \wedge F(S)_j = 0)] = 1 - \frac{1}{2} \cdot 2^{-k} = 1 - 2^{-(k+1)}$$

The result follows from independence across m bit positions. □

Theorem 19 (Subset False Positive Rate)**.** *For sets S_1, S_2 of sizes k_1, k_2, the false positive rate for the subset test $S_1 \subseteq S_2$ is:*

$$\varepsilon_{\subseteq}(k_1, k_2, m) = \left(1 - (1 - 2^{-k_1}) \cdot 2^{-k_2}\right)^m$$

Proof. For $S_1 \subseteq S_2$ to test positive falsely, every bit set in $F(S_1)$ must be set in $F(S_2)$. Bit j is set in $F(S_1)$ with probability $1 - 2^{-k_1}$ and unset in $F(S_2)$ with probability 2^{-k_2}. The probability that bit j does not refute the subset relation is $1 - (1 - 2^{-k_1}) \cdot 2^{-k_2}$, and the result follows from independence. □

Theorem 20 (Space Complexity)**.** *To maintain a fixed false positive rate ε for membership queries on a set of size k, the vector width must satisfy:*

$$m = \mathcal{O}(2^k)$$

This exponential growth limits the single-level scheme to small sets (typically $k \leq 20$).

Proof. From Theorem 18, we have $\varepsilon = (1 - 2^{-(k+1)})^m$. Solving for m:

$$m = \frac{\log \varepsilon}{\log(1 - 2^{-(k+1)})} \approx \frac{\log \varepsilon}{-2^{-(k+1)}} = -\log(\varepsilon) \cdot 2^{k+1}$$

Thus $m = \mathcal{O}(2^k)$ for fixed ε. □

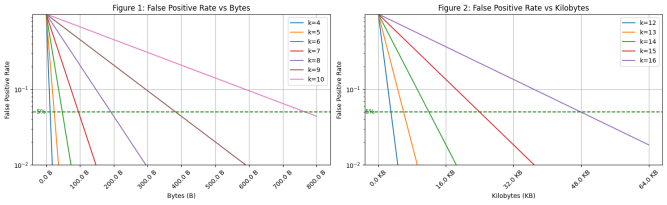

Fig. 2: False positive rate vs. bit-vector width for different set sizes k. Left: small sets ($k = 4$–10) require bytes to achieve 5% FPR. Right: larger sets ($k = 12$–16) require kilobytes, demonstrating the $\mathcal{O}(2^k)$ space complexity.

Remark 21 (Information-Theoretic Consistency)**.** *This exponential growth in k is consistent with the information-theoretic lower bound of $-\log_2(\varepsilon)$ bits per element from the Bernoulli entropy analysis. For the trapdoor Boolean algebra (online construction), the space budget m is fixed at construction time, forcing ε to degrade as k grows. Equivalently, each inserted element consumes $\mathcal{O}(1)$ bits of the fixed-size representation, leaving fewer bits to distinguish non-members, which is why ε increases monotonically with the set size k for fixed m.*

Theorem 22 (Complement Non-Preservation)**.** *The homomorphism F does not preserve complement: $F(\neg_A x) \neq \neg_B F(x)$ for finite sets x.*

Proof. For any finite set x, the complement $\neg_A x = X^* \setminus x$ contains infinitely many elements from the free semigroup X^*. Since the hash function $h : X^* \to \{0,1\}^m$ uniformly distributes over 2^m possible outputs, by the pigeonhole principle, $F(\neg_A x)$ will have all bit positions set to 1 as elements from $\neg_A x$ are added:

$$F(\neg_A x) = 0^m \,|\, h(y_1) \,|\, h(y_2) \,|\, \cdots = 1^m$$

for any enumeration $\{y_1, y_2, \ldots\} \subseteq \neg_A x$.

However, $\neg_B F(x) = \sim(h(x_1) \,|\, \cdots \,|\, h(x_k))$ for finite $x = \{x_1, \ldots, x_k\}$. Since $F(x)$ is a finite OR of k hash values, some bit positions will remain 0 in $F(x)$, making those positions 1 in $\neg_B F(x)$ but also making some positions 0. Thus $\neg_B F(x) \neq 1^m = F(\neg_A x)$. □

Remark 23 (Structural Necessity of Complement Failure)**.** *This asymmetry—AND/OR preserved (at least approximately), NOT not preserved—is structural, not an artifact of the hash construction. Any approximate Boolean algebra homomorphism from an infinite source algebra to a finite target algebra must fail to preserve complement, since the complement of a finite set in an infinite domain covers all hash values by pigeonhole. More precisely, for any finite codomain of size 2^m and any finite set x with $|\neg_A x| = \infty$, the image $F(\neg_A x)$ saturates to the maximal element 1^m, while $\neg_B F(x)$ is strictly submaximal whenever $F(x) \neq 0^m$. This obstruction is independent of the choice of hash function or encoding scheme.*

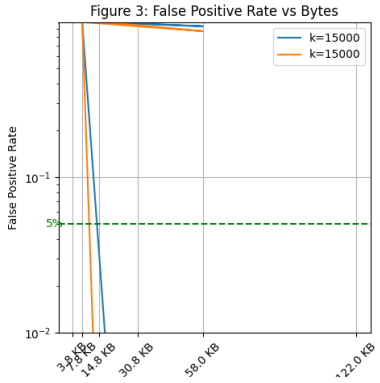

Fig. 3: Two-level hashing achieves practical FPR for large sets ($k = 15,000$) with approximately 60KB storage.

Remark 24 (Boolean Ring Alternative)**.** *An alternative construction uses the Boolean ring* $(\{0,1\}^m, \oplus, \wedge, \mathrm{id}, 0^m)$ *with symmetric difference (XOR) instead of union. This construction maps sets to hash values via* $G(\{x_1, \ldots, x_k\}) = h(x_1) \oplus \cdots \oplus h(x_k)$*. The ring structure supports* only *equality predicates—membership, subset, and complement operations are not available. However, equality testing has optimal FPR of* 2^{-m} *independent of set size, and the output distribution is perfectly uniform, providing stronger privacy guarantees for equality-only applications.*

E. Two-Level Hashing for Scalability

The exponential space complexity of single-level hashing (Theorem 20) motivates a hierarchical approach. The two-level scheme partitions elements into bins, reducing effective set size per bin.

Definition 25 (Two-Level Hash Structure)**.** *Given hash output size* q *bits, partition into:*

- *First* w *bits: bin index (selecting one of* 2^w *bins)*
- *Remaining* $q - w$ *bits: element representation within bin*

Total space: $2^w(q-w)$ *bits. Expected elements per bin:* $k/2^w$*.*

Theorem 26 (Two-Level FPR)**.** *The membership FPR for the two-level scheme with* k *elements is:*

$$\varepsilon(k, w, q) = \left(1 - 2^{-(k/2^w+1)}\right)^{q-w}$$

This achieves practical FPR for large sets: with $w = 8$, $q = 256$, and $k = 1000$, we have approximately 4 elements per bin and FPR $\approx 2^{-12}$, using about 7.8 KB total storage.

F. Unified Hash Construction

All the trapdoor set operations derive from a general hash-based construction that unifies probabilistic data structures with cryptographic privacy [5], [8]:

Definition 27 (Hash-Based Bernoulli Type)**.** *A hash-based Bernoulli type* $\mathcal{B}\langle T\rangle$ *over base type* T *consists of:*

1) *A keyed hash function* $h : \{0,1\}^* \times \{0,1\}^s \to \{0,1\}^m$
2) *For each output value* y*, a set of valid encodings* $\mathsf{Valid}(y) \subseteq \{0,1\}^m$
3) *A seed* s *such that for all inputs* x*:* $h(\mathsf{enc}(x), s) \in \mathsf{Valid}(f(x))$

The resulting type is a Bernoulli approximation *of* T*: operations on* $\mathcal{B}\langle T\rangle$ *approximate operations on* T *with explicit error rates.*

The false positive rate emerges from encoding set sizes: $\alpha = |\mathsf{Valid}(\mathsf{true})|/2^m$. This unified view reveals that Bloom filters, trapdoor functions, and approximate maps are all instances of the same construction with different encoding strategies:

- **Bloom filter**: Multiple hash functions create overlapping valid encoding regions, yielding $\alpha = (1 - e^{-kn/m})^k$
- **Trapdoor set**: Single keyed hash with $\mathsf{Valid}(\mathsf{true}) = \{H(k\|v)\}$, yielding $\alpha = 2^{-m}$ (collision probability)
- **Approximate maps**: Encoding sizes vary by output value, enabling homophonic (frequency-proportional) allocation for privacy

G. Privacy-Space Trade-off

The same hash construction achieves fundamentally different goals by varying encoding sizes [5]:

Space-Optimal Encoding: $|\mathsf{Valid}(y)| = 1$ (one encoding per value). Minimizes the total number of valid cipher values, but the single representation of each y is then observed with that value's own frequency, revealing the frequency distribution. Used in compression and space-efficient data structures.

Privacy-Optimal Encoding: $|\mathsf{Valid}(y)| \propto \mathrm{freq}(y)$ (proportional to frequency). Frequent values receive proportionally more encodings, so under the real query distribution each individual cipher value is equiprobable and the observed cipher-value distribution is flat, hiding frequency patterns. This is the classical Simmons homophonic prescription [9], the key insight connecting approximation to privacy.

Theorem 28 (Uniformity from Homophonic Allocation)**.** *Let each output value* y *occur with probability* $\mathit{freq}(y)$ *and be encoded by a uniform choice within* $\mathsf{Valid}(y)$*. With encoding-set sizes* $|\mathsf{Valid}(y)| = c\,\mathit{freq}(y)$ *for constant* c*, every cipher value* $v \in \mathsf{Valid}(y)$ *is observed with probability*

$$P[\textit{cipher value} = v] = \frac{\mathit{freq}(y)}{|\mathsf{Valid}(y)|} = \frac{1}{N}, \qquad N := \sum_y |\mathsf{Valid}(y)| = c,$$

so the observed cipher-value distribution is uniform without explicit randomization. (The inverse allocation $|\mathsf{Valid}(y)| \propto 1/\mathit{freq}(y)$ *sharpens the distribution instead, the opposite of the privacy goal.)*

The trapdoor set uses uniform-sized encodings (a middle ground), providing practical privacy at constant space cost per element. Each element receives a fixed-size hash representation, offering computational privacy when the input domain has sufficient entropy.

H. Cardinality Estimation

The trapdoor set incorporates the well-established HyperLogLog algorithm [10] for cardinality estimation on hash-transformed sets. We apply HyperLogLog without modification, relying on its proven accuracy guarantees:

Algorithm 1 Cardinality Estimation

Require: Trapdoor set S
Ensure: Estimated cardinality $\hat{n}$
1: $m \leftarrow$ number of buckets
2: $M \leftarrow$ array of m registers
3: **for** each trapdoor $t \in S$ **do**
4: $\quad j \leftarrow$ first $\log_2 m$ bits of t
5: $\quad w \leftarrow$ remaining bits of t
6: $\quad M[j] \leftarrow \max(M[j], \rho(w))$
7: **end for**
8: $\hat{n} \leftarrow \alpha_m \cdot m^2 / \sum_{j=1}^{m} 2^{-M[j]}$
9: **return** $\hat{n}$

The algorithm achieves relative error $1.04/\sqrt{m}$ using $O(m \log \log n)$ bits.

V. Implementation

We provide a reference implementation as the `TrapdoorSet` type in the `trapdoor-maps` library [8], which backs the framework's empirical claims. A trapdoor set stores the element encodings $T_k(v) = H(k\|v)$ (keyed via HMAC-SHA256 over a per-set seed) and exposes:

- the literal-set operations of Section IV-A: union, intersection, difference, and symmetric difference on the encodings, and membership by encoding equality returning a plaintext boolean;
- a seed-derived compatibility tag, so that operations between sets encoded under different seeds are rejected rather than silently producing nonsense;
- the collision-bounded false-positive rate $|S|\,2^{-8b}$ for b-byte encodings, with no false negatives.

Deterministic ($K = 1$) encoding is exactly what makes the untrusted-side equality test possible; the same library realizes the opaque ($K > 1$) regime separately, behind a trusted decoder. The implementation carries unit, property-based, and integration tests, including an end-to-end private-set-intersection example in which an untrusted party returns an opaque intersection that the holder of the seed then decodes.

VI. Evaluation

A. Complexity Analysis

The asymptotic complexity of the trapdoor set operations is determined by the underlying hash computation and set operations:

TABLE I: Operation Complexity

Operation	Time	Space
Trapdoor creation	$O(1)$	$O(n)$ bits
Set membership test	$O(1)$ expected	$O(1)$
Set intersection (k_1, k_2)	$O(\min(k_1, k_2))$	$O(\min(k_1, k_2))$
Set union (k_1, k_2)	$O(k_1 + k_2)$	$O(k_1 + k_2)$

All operations are dominated by hash computation cost: each reduces to a small number of cryptographic hash evaluations and ordinary set operations, with no homomorphic evaluation and no interaction. This is the structural source of the efficiency advantage over fully homomorphic encryption and secure multi-party computation; we do not report wall-clock benchmarks here.

B. Security Evaluation

The security of the trapdoor set rests on the collision resistance and preimage resistance of the underlying hash function. For SHA-256, the expected collision probability is 2^{-256} per pair, providing a negligible false positive rate for practical set sizes.

C. Limitations and Attack Vectors

The trapdoor set provides practical privacy but has important limitations that users must understand:

Dictionary Attacks: For low-entropy input domains (e.g., phone numbers, SSNs, email addresses), an adversary with knowledge of the domain can precompute hashes for all possible values and match against observed trapdoors. Privacy degrades to $O(1/|D|)$ where $|D|$ is the domain size. *Mitigation*: Use high-entropy inputs, apply key stretching (e.g., Argon2), or combine with additional randomization.

Frequency Leakage: Query frequencies are preserved through hash transformations. An adversary observing repeated queries learns which trapdoors correspond to frequently-accessed values. *Mitigation*: Add dummy queries, use query batching, or apply differential privacy noise.

Correlation Leakage: Deterministic hashing reveals equality patterns—repeated queries on the same value produce identical hashes. This leaks that certain queries target the same underlying value. *Mitigation*: Periodic key rotation, though this complicates system design.

Quantum Vulnerability: Grover's algorithm reduces effective hash security from n bits to $n/2$ bits. For quantum resistance, use 512-bit hashes to maintain 256-bit security.

Remark 29 (When the Trapdoor Set Is Appropriate)**.** *The trapdoor set is suitable when: (1) input domains have high entropy, (2) dictionary attacks are infeasible, (3) frequency patterns are acceptable leakage, or (4) approximate plausible deniability suffices. For stronger guarantees, consider FHE or MPC despite their performance costs.*

D. Comparison with Alternatives

TABLE II: Comparison with Related Systems

System	Privacy Model	Dictionary Resistant	Performance	Accuracy
Trapdoor set	Preimage	No*	hashing	Approximate
FHE	Semantic	Yes	s	Exact
MPC	Simulation	Yes	ms	Exact
Bloom Filters	None	N/A	μs	Approximate

*Vulnerable when input domain has low entropy

The trapdoor set occupies a practical niche: preimage-based non-recovery with equality-channel leakage, at the cost of hashing and set operations only, suitable for high-entropy domains where dictionary attacks are infeasible.

VII. Applications

A. Private Set Intersection

The trapdoor set enables efficient PSI without revealing non-matching elements. Using hash-table based intersection, the trapdoor set achieves $O(n)$ complexity with $O(n)$ space, matching the best non-private approaches. The key advantage is that operations occur entirely in the hash domain—the false positive rate for membership is bounded by hash collision probability, while the underlying values remain computationally hidden.

B. Secure Deduplication

Cloud storage providers can identify duplicate files without accessing content:

Algorithm 2 Secure Deduplication

Require: File F, Trapdoor factory T
1: $chunks \leftarrow$ split F into blocks
2: $hashes \leftarrow$ empty set
3: **for** each chunk $c \in chunks$ **do**
4: $\quad h \leftarrow T.create(c)$
5: $\quad$ add h to $hashes$
6: **end for**
7: Query storage for existing $hashes$
8: Upload only unique chunks

C. Privacy-Preserving Analytics

The trapdoor set supports various analytical operations on hash-transformed data:

Histogram Generation: Count occurrences without revealing underlying values, useful for distribution analysis while preserving privacy.

Frequency Analysis: Identify common patterns in hash-transformed data with collision-bounded error rates.

Similarity Metrics: Compute Jaccard similarity and other set-based metrics on private sets with explicit error quantification.

VIII. Related Work

A. Homomorphic Encryption

Fully homomorphic encryption (FHE) enables arbitrary computation on encrypted data. Since Gentry's breakthrough [1], significant progress has been made in practical FHE systems. TFHE [11] and CKKS [12] achieve sub-second bootstrapping, while Brakerski's FHE without modulus switching [13] simplifies implementation. Partially homomorphic schemes like Paillier [14] offer better performance but support only specific operations. The trapdoor set provides a complementary approach, trading exact homomorphic computation for set operations at hashing speed.

B. Secure Multi-Party Computation

MPC protocols enable joint computation without revealing inputs. Classical protocols [2], [15] laid theoretical foundations, while modern frameworks have made MPC practical. MP-SPDZ [16] provides a comprehensive toolkit supporting multiple protocols, while ABY3 [17] achieves efficient three-party computation. Recent advances in function secret sharing [18] have reduced communication complexity. However, MPC still requires multiple rounds of interaction and coordination between parties. The trapdoor set operates non-interactively using only hash transformations, trading exact computation for practical single-party performance.

C. Private Set Intersection

PSI protocols have evolved significantly since early work [19], [20]. Modern protocols achieve remarkable efficiency: OPRF-based PSI [21] handles billion-element sets, while circuit-PSI [22] enables arbitrary computations on intersection results. Unbalanced PSI [23] optimizes for asymmetric set sizes common in practice. The trapdoor set provides a simpler alternative where approximate results are acceptable, requiring only hash computation without cryptographic protocols.

D. Approximate Data Structures

Probabilistic data structures trade accuracy for efficiency. Bloom filters [6] pioneered this approach for membership testing. Cuckoo filters [24] improve on Bloom filters by supporting deletions. Count-Min sketches [25] and streamed cardinality estimation [26] enable frequency and cardinality estimation. Recent work includes learned Bloom filters [27] using machine learning to optimize performance, and XOR filters [28] achieving near-optimal space efficiency. The trapdoor set builds upon these foundations, adding cryptographic privacy through hash transformations while maintaining the efficiency benefits of approximate operations.

E. Differential Privacy

Differential privacy (DP) [3] provides statistical privacy guarantees through calibrated noise addition. The field has matured significantly with deployment at major tech companies [29], [30]. Recent advances include mechanism design via differential privacy [31], concentrated DP [32] for tighter privacy accounting, and shuffle DP [33] amplifying privacy through anonymization. Practical secure aggregation [34] enables federated learning at scale. The trapdoor set provides complementary cryptographic privacy that can be composed with DP techniques, offering defense-in-depth for sensitive applications.

IX. Conclusion

We presented the trapdoor set, a value type for privacy-preserving set operations over deterministic one-way encodings. It supports a collision-bounded generalized Boolean algebra over opaque elements, computed by an untrusted party that never decodes; its cost is bounded by standard hashing

and set operations, and its confidentiality is governed by the equality channel, tunable through the homophony parameter that connects it to the cipher-map framework [4].

Our contributions include:

- A systematic framework for error propagation through composed set operations on hash-transformed data
- Integration of established probabilistic algorithms (Bloom filters, HyperLogLog) with cryptographic privacy guarantees
- A reference Python implementation with operations reducible to standard hash computations
- A worked private-set-intersection example exercised by the reference test suite

Future directions include:

- Integration with post-quantum hash functions for quantum resistance
- Hardware acceleration via AES-NI and SHA extensions
- Composition with differential privacy for enhanced protection
- Formal verification of implementation correctness

A. Current Limitations

The current Python reference implementation provides core trapdoor and set operations with full error propagation. The following features are not yet implemented:

- **Cryptographic key derivation**: The implementation uses direct SHA-256 concatenation ($H(k\|v)$); production deployment would benefit from HKDF or Argon2 for proper key derivation
- **Differential privacy integration**: Combining the trapdoor set with calibrated noise addition is a natural extension but not yet implemented
- **Constant-time primitives**: The Python implementation does not guarantee constant-time execution; side-channel resistance would require a lower-level language
- **Performance optimization**: The reference implementation prioritizes clarity over performance; a production implementation in C or Rust would benefit from SIMD hashing and memory pooling

The trapdoor set demonstrates that practical privacy-preserving computation is achievable when applications can accept approximate results with explicit error bounds.

References

[1] C. Gentry, "Fully homomorphic encryption using ideal lattices," in *Proceedings of the 41st annual ACM symposium on Theory of computing*, 2009, pp. 169–178.

[2] A. C.-C. Yao, "Protocols for secure computations," in *23rd Annual Symposium on Foundations of Computer Science*. IEEE, 1982, pp. 160–164.

[3] C. Dwork, F. McSherry, K. Nissim, and A. Smith, "Calibrating noise to sensitivity in private data analysis," in *Theory of Cryptography Conference*. Springer, 2006, pp. 265–284.

[4] A. Towell, "Cipher maps: A unified framework for oblivious function evaluation," 2024, working paper.

[5] ——, "Bernoulli types: A framework for approximate computation," 2024, working paper.

[6] B. H. Bloom, "Space/time trade-offs in hash coding with allowable errors," *Communications of the ACM*, vol. 13, no. 7, pp. 422–426, 1970.

[7] R. Curtmola, J. Garay, S. Kamara, and R. Ostrovsky, "Searchable symmetric encryption: improved definitions and efficient constructions," in *Proceedings of the 13th ACM Conference on Computer and Communications Security (CCS)*, 2006, pp. 79–88.

[8] A. Towell, "Trapdoor computing: Privacy through uniform encoding," 2024, working paper.

[9] G. J. Simmons, "Symmetric and asymmetric encryption," *ACM Computing Surveys*, vol. 11, no. 4, pp. 305–330, 1979.

[10] P. Flajolet, É. Fusy, O. Gandouet, and F. Meunier, "Hyperloglog: the analysis of a near-optimal cardinality estimation algorithm," in *Conference on Analysis of Algorithms*, 2007, pp. 127–146.

[11] I. Chillotti, N. Gama, M. Georgieva, and M. Izabachène, "Tfhe: Fast fully homomorphic encryption over the torus," *Journal of Cryptology*, vol. 33, no. 1, pp. 34–91, 2020.

[12] J. H. Cheon, A. Kim, M. Kim, and Y. Song, "Homomorphic encryption for arithmetic of approximate numbers," in *International Conference on the Theory and Application of Cryptology and Information Security*, 2017, pp. 409–437.

[13] Z. Brakerski, "Fully homomorphic encryption without modulus switching from classical GapSVP," in *Annual Cryptology Conference (CRYPTO)*. Springer, 2012, pp. 868–886.

[14] P. Paillier, "Public-key cryptosystems based on composite degree residuosity classes," in *International Conference on the Theory and Application of Cryptographic Techniques*. Springer, 1999, pp. 223–238.

[15] O. Goldreich, S. Micali, and A. Wigderson, "How to play any mental game," in *Proceedings of the 19th Annual ACM Symposium on Theory of Computing*, 1987, pp. 218–229.

[16] M. Keller, "Mp-spdz: A versatile framework for multi-party computation," in *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, 2020, pp. 1575–1590.

[17] P. Mohassel and P. Rindal, "Aby3: A mixed protocol framework for machine learning," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, 2018, pp. 35–52.

[18] E. Boyle, N. Chandran, N. Gilboa, D. Gupta, Y. Ishai, N. Kumar, and M. Rathee, "Function secret sharing for mixed-mode and fixed-point secure computation," in *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, 2021, pp. 871–900.

[19] C. Meadows, "A more efficient cryptographic matchmaking protocol for use in the absence of a continuously available third party," *Proceedings of IEEE Symposium on Security and Privacy*, pp. 134–137, 1986.

[20] M. J. Freedman, K. Nissim, and B. Pinkas, "Efficient private matching and set intersection," in *International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 2004, pp. 1–19.

[21] S. Raghuraman and P. Rindal, "Blazing fast psi from improved okvs and subfield vole," in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, 2022, pp. 2505–2517.

[22] N. Chandran, D. Gupta, and A. Shah, "Circuit-psi with linear complexity via relaxed batch opprf," in *Proceedings on Privacy Enhancing Technologies*, vol. 2022, no. 1, 2022, pp. 353–372.

[23] H. Chen, K. Laine, and P. Rindal, "Fast private set intersection from homomorphic encryption," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, 2017, pp. 1243–1255.

[24] B. Fan, D. G. Andersen, M. Kaminsky, and M. D. Mitzenmacher, "Cuckoo filter: Practically better than bloom," in *Proceedings of the 10th ACM International on Conference on emerging Networking Experiments and Technologies*, 2014, pp. 75–88.

[25] G. Cormode and S. Muthukrishnan, "An improved data stream summary: the count-min sketch and its applications," *Journal of Algorithms*, vol. 55, no. 1, pp. 58–75, 2005.

[26] D. Ting, "Streamed approximate counting of distinct elements: Beating optimal batch methods," in *Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2014, pp. 442–451.

[27] T. Kraska, A. Beutel, E. H. Chi, J. Dean, and N. Polyzotis, "The case for learned index structures," in *Proceedings of the 2018 International Conference on Management of Data*, 2018, pp. 489–504.

[28] T. M. Graf and D. Lemire, "Xor filters: Faster and smaller than bloom and cuckoo filters," *Journal of Experimental Algorithmics*, vol. 25, pp. 1–16, 2020.

[29] Ú. Erlingsson, V. Pihur, and A. Korolova, "Rappor: Randomized aggregatable privacy-preserving ordinal response," in *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security*, 2014, pp. 1054–1067.

[30] A. D. P. Team, "Learning with privacy at scale," *Apple Machine Learning Journal*, vol. 1, no. 8, 2017.

[31] F. McSherry and K. Talwar, "Mechanism design via differential privacy," in *48th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*. IEEE, 2007, pp. 94–103.

[32] M. Bun and T. Steinke, "Concentrated differential privacy: Simplifications, extensions, and lower bounds," in *Theory of Cryptography Conference*, 2016, pp. 635–658.

[33] Ú. Erlingsson, V. Feldman, I. Mironov, A. Raghunathan, K. Talwar, and A. Thakurta, "Amplification by shuffling: From local to central differential privacy via anonymity," in *Proceedings of the Thirtieth Annual ACM-SIAM Symposium on Discrete Algorithms*, 2019, pp. 2468–2479.

[34] K. Bonawitz, V. Ivanov, B. Kreuter, A. Marcedone, H. B. McMahan, S. Patel, D. Ramage, A. Segal, and K. Seth, "Practical secure aggregation for privacy-preserving machine learning," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, 2017, pp. 1175–1191.